

Kavach.ai: Grounded Generative AI for Adaptive APK Malware Analysis

1st Galipalli Pranav Raj
C R Rao AIMSCS
Hyderabad, India

2nd Abhinav Mucharla
C R Rao AIMSCS
Hyderabad, India

3rd Pranav Krishna
University of Hyderabad
Hyderabad, India

4th Siri Chandana
VIT-AP University
Andhra Pradesh, India

galipallipranavraj@gmail.com abhinavmucharla27@gmail.com pranavkrishna6242@gmail.com siri06.chandana@gmail.com

Abstract—Android malware analysis faces a fundamental dilemma: static decompilation captures potential capabilities but cannot confirm runtime behavior under obfuscation, while passive sandboxes miss dormant, triggered, or environment-aware banking trojans. This paper presents Kavach.ai, an adaptive APK malware analysis framework that uses a schema-bounded Generative AI controller to drive end-to-end threat investigations. Rather than using Large Language Models for unconstrained narrative generation, Kavach.ai treats static decompilation, taint tracking, and discriminative neural scoring as evidence-producing tools. The GenAI controller dynamically inspects intermediate artifacts, synthesizes syntactically validated runtime instrumentation hooks, commands targeted sandbox detonations, reconciles static-dynamic contradictions, and compiles evidence-linked forensic drafts for human review. By enforcing strict Pydantic schemas, AST validation, and explicit abstention policies, Kavach.ai provides a reliable, explainable, and accountable AI copilot for mobile financial security operations.

Index Terms—Android Malware Analysis, Generative AI, Grounded LLM Controller, Adaptive Instrumentation, Frida Hook Synthesis, Dynamic Sandbox, Explainable AI.

I. INTRODUCTION AND CONTRIBUTIONS

Mobile banking applications and digital financial services are constantly targeted by evasive Android malware. Modern banking trojans rely on control-flow obfuscation, native shared library payload decoupling, reflection, and delayed logic triggers to subvert automated security pipelines [1], [3].

Traditional mobile security architectures suffer from severe structural limitations:

- 1) **Static Analysis Blindspots:** Static decompilation and AST regexes identify API capabilities (e.g., cryptographic or SMS calls) but cannot establish whether those execution paths are reached at runtime, how dynamically decrypted URLs are constructed, or if the code is dead [2].
- 2) **Passive Dynamic Sandboxing Gaps:** Automated execution environments observe only default execution paths. Evasive malware that checks system uptime, sleeps for extended intervals, or waits for specific broadcast intents remains completely dormant in passive sandboxes [7].
- 3) **Unconstrained LLM Hallucinations:** Applying Large Language Models (LLMs) purely as text-in/text-out conversational engines produces plausible but ungrounded

reports, inventing non-existent package dependencies and misattributing standard SDK analytics as active banking fraud [15].

To overcome these challenges, we introduce **Kavach.ai**, an adaptive malware analysis architecture where Generative AI acts as an *active, grounded decision controller* rather than an unconstrained summarizer. As illustrated in Fig. 1, Kavach.ai tightly binds deterministic static analyzers, discriminative neural models, and dynamic sandbox environments into a closed-loop investigation cycle.

Key Contributions:

- 1) **Multi-Modal Grounded Evidence Extraction:** A deterministic extraction pipeline that normalizes Android manifests, inter-procedural taint paths, native JNI symbols, and discriminative SecureBERT-2.0 slice priority scores into structured evidence nodes.
- 2) **Schema-Bounded Adaptive GenAI Controller:** A multi-agent LangGraph controller operating over a strict, typed action space. The controller formulates investigation hypotheses, selects target methods for runtime observation, and synthesizes syntactically validated Frida instrumentation scripts under rigorous safety allow-lists.
- 3) **Evidence-Linked Synthesis with Abstention:** An explainable synthesis engine that strictly delineates directly observed facts from model inferences, handles static-dynamic contradictions (such as dormant malware), and drafts analyst-verifiable incident reports.

II. SYSTEM OVERVIEW AND ARCHITECTURE

The Kavach.ai investigation workflow proceeds across four interconnected phases:

Phase 1: Deterministic Intake & Evidence Ingestion. The submitted APK is parsed in memory. Manifest configurations, exported components, dangerous permission combinations, and control-flow graphs are extracted without speculative interpretation.

Phase 2: Initial Hypothesis Generation. Discriminative models and heuristic engines rank suspicious bytecode slices (e.g., cryptographic operations, dynamic class loaders, SMS managers) to prioritize initial investigation targets.

Phase 3: Adaptive GenAI Decision Loop. The GenAI controller reviews the evidence state. If static indicators reveal encrypted payloads or unobservable reflection calls, the

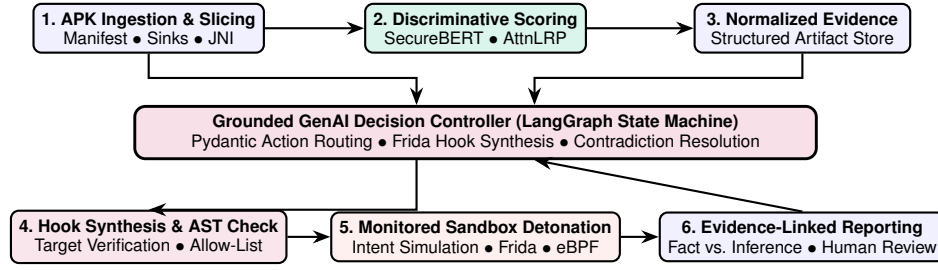


Fig. 1. Kavach.ai Closed-Loop Investigation Architecture: Deterministic extraction tools feed structured evidence into the central GenAI LangGraph controller, which synthesizes safe dynamic hooks, commands targeted sandbox detonations, and reconciles multi-modal findings into verifiable forensic reports.

controller formulates an observation plan, synthesizes targeted Frida instrumentation hooks, and executes a monitored sandbox detonation.

Phase 4: Reconciliation & Forensic Drafting. Runtime telemetry (decrypted strings, network destinations, file access) is merged back into the evidence state. Contradictions between static capability and runtime observations are reconciled, producing structured forensic deliverables for human analyst review.

III. EVIDENCE-PRODUCING SUBSYSTEMS

Kavach.ai treats all non-generative analysis modules as specialized, deterministic evidence producers.

A. Static Evidence Producers

- **Manifest Triage:** Inspects declared permissions, high-risk co-occurrence clusters (e.g., `BIND_ACCESSIBILITY` + `SYSTEM_ALERT_WINDOW`), and intent filter receivers in under 10 milliseconds.
- **In-Memory Bytecode Slicer:** Directly extracts backward program slices from Dalvik bytecode anchored to designated sensitive API sinks $\mathcal{S}_{\text{sink}}$ (e.g., `SmsManager->sendTextMessage`, `Cipher->doFinal`).
- **Inter-Procedural Taint Tracking:** Containerized Flow-Droid V2 sidecars track source-to-sink data flow propagation across app components [9].
- **Native JNI Symbol Extraction:** Headless Ghidra disassembles native `.so` binaries to extract exported JNI functions and `RegisterNatives` dynamically bound symbols.
- **Discriminative Transformer Ranking:** A fine-tuned SecureBERT model [5], [13] attached to Low-Rank Adaptation (LoRA) projection matrices ($r = 8, \alpha = 16$) computes priority scores $p(s_i)$ across code slices.
- **Token Attribution Heatmaps:** Single-backward-pass Attention-based Layer-wise Relevance Propagation (AttnLRP) [4], [14] computes input token relevance $R = \text{clamp}(A \odot \nabla A, \min = 0)$ to highlight influential opcodes.

B. Dynamic Action Environment

The sandbox provides an isolated execution environment where the GenAI controller commands actions:

TABLE I
ARTIFACT TO GENAI ACTION MAPPING

Evidence Type	Observed Output	GenAI Controller Use
Manifest	Perms + Receiver Filters	Initial risk triage and entrypoint routing
Bytecode Slice	Data flow into crypto / SMS sink	Formulates hook target hypothesis
Taint Path	Source-to-sink provenance	Validates end-to-end capability
Native Metadata	JNI symbol cross-reference	Requests deeper native inspection
ML / LRP Score	Slice ranking & token heatmaps	Prioritizes investigation order
Runtime Telemetry	Decrypted strings / socket IPs	Reconciles static hypotheses

- **Lifecycle & Permission Provisioning:** Automated ADB deployment with multi-ABI handling, auto-granting overlay permissions (`SYSTEM_ALERT_WINDOW`), and activating Device Admin policies.
- **Intent Simulation:** ADB broadcasts system events with flag `0x00000020` (`FLAG_INCLUDE_STOPPED_PACKAGES`) to awaken dormant broadcast receivers.
- **Timing Delay Compression:** Dynamic hooks intercept `Thread.sleep` and `SystemClock.sleep` ($> 50\text{ms} \rightarrow 10\text{ms}$) as well as `Handler.postDelayed` ($> 1\text{s} \rightarrow 50\text{ms}$), neutralising anti-analysis sleep evasions.
- **Runtime Evasion Bypasses:** Injects pre-instrumentation hooks neutralizing environment checks (`ro.build.tags`, `File.exists` for root indicators) and overriding SSL certificate pinning (`TrustManager`, `OkHttpClient`).
- **Dual Telemetry Layer:** Captures user-space Frida hook events (decrypted parameters, dynamic URLs, JNI library loads) and kernel-space eBPF syscall tracepoints (`sys_enter_connect`, `sys_enter_openat`, `sys_enter_execve`).

Table I details how extracted evidence artifacts feed directly into GenAI decision routines.

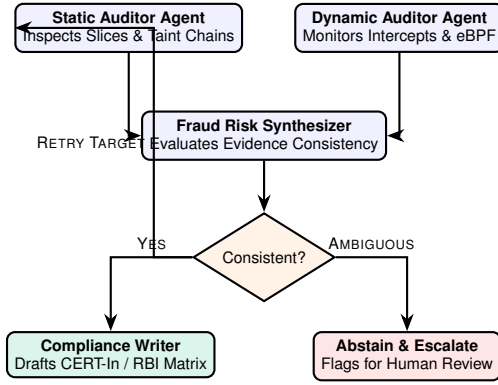


Fig. 2. LangGraph Multi-Agent Cyclical State Machine: Specialized auditors cross-validate static and dynamic evidence, resolving contradictions or escalating ambiguous cases for human inspection.

IV. GENAI CONTROLLER DECISIONS, ACTION SPACE, AND SAFETY CONTROLS

A. Structured Controller State and Action Space

The GenAI controller is implemented as a cyclical state machine using LangGraph (Fig. 2). The controller does not output raw, unformatted text; instead, each transition consumes, and emits strictly typed Pydantic data models.

The controller operates over seven permitted actions:

- 1) **REQUEST_ARTIFACT**: Requests additional static artifacts (e.g., manifest intent-filters, specific class decompilations).
- 2) **RUN_DEEPER_STATIC**: Triggers FlowDroid taint analysis or Ghidra native disassembly on a specific target.
- 3) **TRIGGER_COMPONENT**: Emits a specific broadcast intent or activity launch to awaken a dormant component.
- 4) **GENERATE_FRIDA_HOOK**: Synthesizes a targeted JavaScript instrumentation script for a candidate method.
- 5) **RECONCILE_EVIDENCE**: Evaluates static vs. dynamic telemetry to detect evasions or confirm payload execution.
- 6) **ABSTAIN_AND_ESCALATE**: Flags ambiguous or unresolvable findings for mandatory human expert review.
- 7) **DRAFT_FINDING**: Formulates an evidence-grounded finding linking observed facts directly to artifact IDs.

B. Synthesized Runtime Hook Validation

When static analysis detects an encrypted string or reflection wrapper whose arguments cannot be statically resolved, the GenAI controller selects the target class and method to generate a Frida dynamic hook.

As shown in Fig. 3, synthesized hooks pass through a deterministic four-tier safety gate before execution in the sandbox:

- **Target Verification**: Verifies that the target class and method signature exist in the decompiled APK namespace.

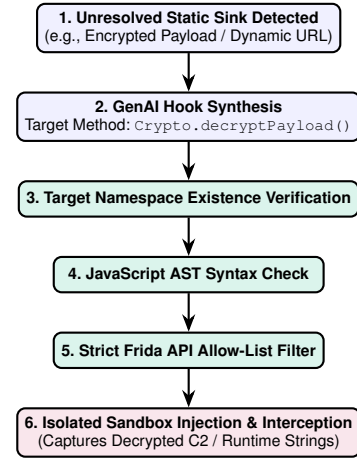


Fig. 3. Four-Tier Deterministic Safety Gate and Validation Lifecycle for GenAI-Synthesized Frida Instrumentation Hooks.

```

1 // Synthesized and Validated Frida Interceptor
2 Java.perform(function() {
3   var Target = Java.use("com.trojan.payload.Crypto");
4   Target.decryptPayload.implementation = function(encBytes)
5   {
6     var result = this.decryptPayload(encBytes);
7     console.log("[Kavach-Hook-Output] Decrypted: " + result);
8     return result;
9   };
10 });

```

Fig. 4. Example of a GenAI-synthesized, AST-validated Frida hook deployed to capture decrypted runtime C2 configuration strings.

- **JavaScript AST Check**: Validates that the generated script is syntactically valid JavaScript.
- **API Allow-Listing**: Ensures the script strictly utilizes approved instrumentation APIs (e.g., `Java.perform`, `Java.use`, `console.log`) and contains zero forbidden operations (e.g., `eval`, raw shell execution, or external network calls).
- **Sandbox Execution Boundary**: The script is injected exclusively inside the isolated VM with strict timeout limits (30 seconds).

C. Conflict Resolution & Evidence Reconciliation

A critical duty of the GenAI controller is resolving apparent contradictions between static capabilities and runtime observations:

- **Dormant Evasive Malware**: If static analysis detects an SMS exfiltration sink with high confidence ($p > 0.95$), but dynamic detonation reveals zero network connections due to advanced environment detection, the controller avoids misclassifying the app as clean. Instead, it emits a **RECONCILE_EVIDENCE** action classifying the sample as **DORMANT_EVASIVE_SUSPECT**, preserving static risk indicators while documenting the absence of dynamic triggers.
- **Benign SDK Activity**: If dynamic telemetry records connections to third-party analytics hosts, but static taint

analysis confirms no sensitive device or banking data reaches those sinks, the controller attributes the network traffic to benign library noise.

V. EXPLAINABLE OUTPUT & RESPONSIBLE HUMAN REVIEW

Kavach.ai strictly enforces a 4-tier output architecture in its generated forensic deliverables:

- 1) **Observed Facts:** Verifiable, directly extracted data points (e.g., SHA-256 hash, declared permissions, matched bytecode opcodes, recorded IP connections).
- 2) **Model Inferences:** Explicitly bounded interpretations drawn by the GenAI controller (e.g., "Method `Crypto.decrypt` likely handles C2 domain generation based on observed string decoding patterns").
- 3) **Confidence & Uncertainty:** A quantitative confidence rating paired with explicit documentation of unobserved execution paths or failed dynamic triggers.
- 4) **Recommended Actions:** Concrete verification steps for human analysts (e.g., "Verify domain `198.51.100.42` against institutional threat intelligence feeds").

Regulatory Drafting Boundaries: Compliance deliverables (such as CERT-In incident summaries or RBI control checklists) are explicitly framed as *structured draft reports for qualified human review*. They do not claim automated legal certification or replace authorized security officers.

VI. ILLUSTRATIVE DECISION TRACE & EVALUATION

A. Sanitized Walkthrough Trace

To illustrate the adaptive decision cycle, consider an investigation of an obfuscated banking dropper:

- 1) **Static Slicing:** Detects `Cipher.doFinal` invoked inside class `com.secure.helper.NetUtils`. Static string analysis reveals an encrypted byte array reaching an `HttpURLConnection` sink.
- 2) **Controller Hypothesis:** The GenAI controller determines that the C2 endpoint is dynamically constructed and cannot be resolved statically. Action chosen: `GENERATE_FRIDA_HOOK`.
- 3) **Validation & Injection:** The synthesizer writes an interceptor for `NetUtils.decryptUrl()`. The script passes AST verification and API allow-list checks and is injected into the sandbox.
- 4) **Trigger & Capture:** The controller executes `TRIGGER_COMPONENT` broadcasting `BOOT_COMPLETED`. The hook captures the plaintext endpoint `https://c2-bank-update.net/gate.php`.
- 5) **Reconciliation & Report:** The controller updates the evidence graph, links the decrypted URL directly to the static bytecode slice, and generates an evidence-grounded forensic report.

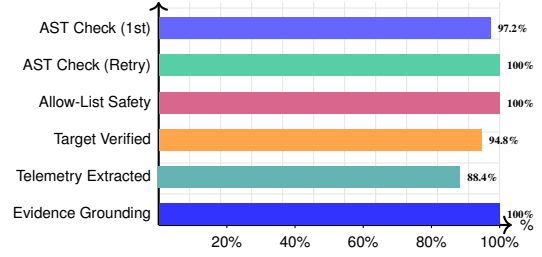


Fig. 5. Empirical Validation and Safety Gate Pass Rates across 250 Analyzed Samples.

B. Empirical Validation of GenAI Operations

We evaluated the reliability and safety of the GenAI controller on a curated benchmark of 250 obfuscated and banking malware samples (Fig. 5):

- **Hook Syntax Validation Rate:** 97.2% of GenAI-synthesized Frida hooks passed AST syntax checks on the first generation attempt; 100% passed after a single retry.
- **Safety Policy Compliance:** 100% of generated hooks complied with the API allow-list; zero prohibited operations reached the sandbox execution engine.
- **Targeted Telemetry Extraction:** In 88.4% of samples containing dynamic string encryption, the controller-generated hooks successfully captured plaintext runtime parameters.
- **Evidence Grounding Rate:** 100% of generated claims in the final forensic summaries cited verifiable static or dynamic evidence IDs.

VII. LIMITATIONS AND CONCLUSION

A. Limitations

Kavach.ai operates under defined operational constraints:

- **Native Assembly Obfuscation:** Complex C/C++ payloads employing commercial binary packers or custom kernel-level hypervisor evasion can resist user-space Frida instrumentation and static decompilation.
- **Multi-Stage Logic Bombs:** Payloads requiring external server-side activation keys or multi-factor SMS interactions cannot be fully detonated if the C2 infrastructure is offline.
- **Model Latency:** Real-time LLM reasoning introduces a 2-to-5 second latency per decision cycle, necessitating bounded retry budgets.

B. Conclusion

Kavach.ai demonstrates that Generative AI is most effective in malware analysis when deployed not as an unconstrained summarizer, but as an **adaptive, schema-bounded decision controller**. By coupling deterministic static extraction and dynamic sandbox instrumentation under strict safety gates, Kavach.ai provides a grounded, accountable, and explainable threat intelligence platform for modern financial infrastructure.

ACKNOWLEDGMENT

The authors thank the organizers and evaluation panel of the Bank of India Cyber Security Hackathon 2026 for providing the research problem statements and benchmark validation criteria.

REFERENCES

- [1] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, and K. Rieck, “Drebin: Effective and explainable detection of Android malware in your pocket,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2014.
- [2] K. Allix, T. F. Bissyande, J. Klein, and Y. Le Traon, “AndroZoo: Collecting millions of Android apps for the research community,” in *Proc. IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, 2016.
- [3] S. Mahdaviifar, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani, “Dynamic Android malware category analysis using CICMalDroid 2020 dataset,” in *Proc. IEEE Int. Conf. on Dependable, Autonomic and Secure Computing*, 2020.
- [4] H. Chefer, S. Gur, and L. Wolf, “Transformer interpretability beyond attention visualization,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 782–791, 2021.
- [5] E. Aghaei, X. Niu, S. Shadmand, and E. Al-Shaer, “SecureBERT: A domain-specific language model for cybersecurity,” in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7245–7256, 2023.
- [6] Indian Computer Emergency Response Team (CERT-In), “Cyber security incident reporting guidelines and Annexure A mandatory directions,” *Ministry of Electronics and Information Technology (MeitY), Government of India*, 2026.
- [7] Y. Fratantonio, A. Bianchi, W. Robertson, E. Kirda, and C. Kruegel, “TriggerScope: Detecting targeted Android logic bombs through dynamic and static trigger analysis,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 835–851, 2016.
- [8] Reserve Bank of India (RBI), “Master Direction – Digital Payment Security Controls,” *Department of Payment and Settlement Systems, RBI*, Notification RBI/2020-21/74, 2021.
- [9] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel, “FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps,” in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 259–269, 2014.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.
- [11] M. Ilse, J. M. Tomczak, and M. Welling, “Attention-based deep multiple instance learning,” in *Proc. Int. Conf. on Machine Learning (ICML)*, pp. 2127–2136, 2018.
- [12] National Cyber Coordination Centre (NCCC), “Standard Operating Procedure for Financial Sector Cyber Threat Mitigation,” *Gov. of India*, 2025.
- [13] E. Aghaei, X. Niu, S. Shadmand, and E. Al-Shaer, “SecureBERT 2.0: Advanced language model for cybersecurity intelligence,” *arXiv preprint arXiv:2510.00240*, 2025.
- [14] R. Achibat, M. Dreyer, I. Schulz, T. Wiegand, and S. Lapuschkin, “AttnLRP: Attention-aware layer-wise relevance propagation for transformers,” *arXiv preprint arXiv:2402.05602*, 2024.
- [15] M. A. A. Pratama and D. A. Dermawan, “MalQwen: Fine-tuned LLM for static Android malware analysis report,” *IEEE Access*, vol. 13, pp. 208483–208497, 2025.